Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Command kalıbı: amaç (ekrandan, 00:10:00)
  - Normalde bir nesneye istek göndermek için onun bir metodunu çağırırız. Command kalıbı metot çağrılarını (istekleri) nesneye çevirir.
  - Neden isteriz: mesajları kuyruğa koymak (asenkron çalışma, yanıtın geç hesaplandığı durumlar), her metot çağrısının günlüğünü (log) tutmak gibi makul istekler. Bunlar kalıp olmadan ya her çağrının önüne/arkasına ek kod yazarak ya da Aspect Oriented Programming gibi kendine özel dil ve sanal makine gerektiren bir yaklaşımla yapılır.
  - GoF örneği: bir GUI kütüphanesinde menü seçenekleri ve alt seçenekleri vardır; en alttaki seçeneğe tıklanınca bir komut yürütülür. Aynı komut bir düğmenin tıklama olayına da bağlanabilir.
  - Örnek şema: `Command` soyut sınıfı `Execute()` metodunu tanımlar. `OpenCommand` bunu gerçekler; `Execute()` içinde `name = AskUser()`, `doc = new Document(name)`, `application->Add(doc)`, `doc->Open()` çalışır. `OpenCommand`, `application` ilişkisiyle `Application`'ı (`Add(Document)`) bilir.
  - Parça-bütün ilişkisinde parçanın bütünü de bilmesi gerekebilir: açma komutu, açtığı dokümanı uygulamanın dokümanlarına ekleyebilmek için uygulamaya geri bağlantıya ihtiyaç duyar. Benzer şekilde bir `PasteCommand` da dokümanın `Paste` metodunu kullanır.
- Command kalıbının yapısı (ekrandan, 00:11:20)

```
Command {Execute()}                          (soyut)
ConcreteCommand {Execute(), state} ──▷ Command        (kalıtım)
ConcreteCommand ──receiver──► Receiver {Action()}
   Execute():  receiver->Action();
Invoker ◇──► Command                         (toplama)
Client ──► Receiver
Client ┈┈► ConcreteCommand                    (bağımlılık)
```

  - Bütün komutlar ortak bir arayüz altında toplanır; bu arayüzde en az bir ortak metot imzası (`Execute`) bulunur. Somut komutlar `Execute` içinde isteğin alıcısının (Receiver) yapması gereken eylemi (`Action`) yaptırır.
  - Receiver her zaman Invoker ile aynı varlık olmayabilir (açma örneğinde aynıydı). Receiver farklı olabilecekse komuta verilmesi gerekir.
  - Receiver'ı `Execute` metoduna parametre olarak vermek, parametrenin kullanılmadığı durumlar olacaksa tasarım zayıflığıdır (aktarılan ama hiç kullanılmayan parametre).
  - Bunun yerine arayüze bir `setReceiver` metodu eklenip somut komutlarda gerçeklenebilir. Arayüz soyut sınıfa çevrilebiliyorsa Receiver yönetim kodu o sınıfa yazılır. Receiver bazı komutlarda gerekip bazılarında gerekmiyorsa, somut komutlara farklı yapıcılar (receiver alan ve almayan) yazılır; bu yalnız oluşturma anında yapıldığı için büyük sakınca değildir.
- Command örneği: akıllı ev kumandası (ekrandan, 00:14:40)
  - Problem: akıllı ev yazılımında basit bir aç/kapa kumandası ekranı var. Ekrandaki aç/kapa düğmelerine odaların ışıklarını, televizyonları, ısıtıcı/soğutucuları açma/kapama işlevleri kazandırılacak. Esinlenme: Head First Design Patterns.

- Kumanda üzerinde yapılacak işler: belli numaralı düğmeyi açmak, düğme eklemek, belli düğmeyi kapatmak, bütün düğmeleri kapatmak.
- Tasarım: «interface» `AcKapa` (`+ ac(): boolean`, `+ kapa(): boolean`). Bunu gerçekleyen `AydinlatmaDugmesi`, `ElektronikAlet`, `Iklimlendirme` sınıfları (her birinde `- isim: String`, yapıcı, `ac`, `kapa`, `toString`). `Kumanda` sınıfı `- acKapaDugmeleri: ArrayList<AcKapa>` tutar ve `ac(dugmeNo)`, `kapa(dugmeNo)`, `dugmeEkle(dugme)`, `hepsiniKapa()` metotlarını sunar. `AnaProgram` bir `Kumanda` tutar.
- Execute yerine iki metot (`ac`, `kapa`) vardır, çünkü yapılacak iki farklı iş vardır. Tek bir `execute` metoduna "aç mı kapa mı" parametresi vermek de olurdu; metot imzasını ihtiyaca göre tanımlamakta sakınca yoktur.
- Farklı somut sınıflar gerekir, çünkü bir aydınlatma düğmesini, bir elektronik aleti ve bir iklimlendirme cihazını açmak gerçekte farklı kodlarla (SCADA ya da cihazların açık API'leri) yapılır.
- `ArrayList` seçimi, belli numaralı düğmeye doğrudan erişim içindir; kalıpla ilgili değildir.
- Kodlar (ekrandan, 00:17:40, 00:19:20, 00:20:00):

```
public interface AcKapa {
    public boolean ac( );
    public boolean kapa( );
}
public class AydinlatmaDugmesi implements AcKapa {
    private String isim;
    public AydinlatmaDugmesi(String isim) { this.isim = isim; }
    public String toString( ) { return "AcKapa:" + isim; }
    public boolean ac() { return true; }
    public boolean kapa() { return true; }
}
public class Kumanda {
    private ArrayList<AcKapa> acKapaDugmeleri;
    public Kumanda( ) { acKapaDugmeleri = new ArrayList<AcKapa>(); }
    public void dugmeEkle( AcKapa dugme ) { acKapaDugmeleri.add(dugme); }
    public boolean ac( int dugmeNo ) {
        if( acKapaDugmeleri.get(dugmeNo) == null )
            return false;
        return acKapaDugmeleri.get(dugmeNo).ac();
    }
    public boolean kapa( int dugmeNo ) {
        if( acKapaDugmeleri.get(dugmeNo) == null )
            return false;
        return acKapaDugmeleri.get(dugmeNo).kapa();
    }
    public void hepsiniKapa( ) {
        for( AcKapa dugme : acKapaDugmeleri )
            dugme.kapa();
    }
}
public class AnaProgram {
    Kumanda kumanda;
    public AnaProgram( ) {
        kumanda = new Kumanda();
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Antre" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Salon" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Mutfak" ) );
        kumanda.dugmeEkle( new AydinlatmaDugmesi( "Oda" ) );
        kumanda.dugmeEkle( new ElektronikAlet( "Vestel TV" ) );
        kumanda.dugmeEkle( new Iklimlendirme( "Kombi" ) );
    }
    public void evBoskenSokakKapisiAcildi( ) {
        kumanda.ac(0); kumanda.ac(5);
```

```
    }
    public void evdeKimseKalmadi( ) { kumanda.hepsiniKapa( ); }
    public static void main(String[] args) {
        AnaProgram prg = new AnaProgram();
        prg.evBoskenSokakKapisiAcildi();
        System.out.println("Evden çıkılıyor");
        prg.evdeKimseKalmadi();
    }
}
```

  - Ev boşken sokak kapısı açılınca 0 numaralı düğme (antre ışığı) ve 5 numaralı düğme (kombi) açılır; evden çıkılınca bütün düğmeler kapanır.
  - Örnekte bir `getState` metodu eksik. Hocanın notu: örnek, bir termometre ölçümünün geri döndürülmesiyle (Receiver'ın bir eyleminin sonuç döndürmesi) geliştirilebilir.
  - Ders sonrası not: Kodda düğmenin olmadığı durum `acKapaDugmeleri.get(dugmeNo) == null` ile denetleniyor. Java'da `ArrayList.get` aralık dışındaki bir indekste `null` döndürmez, `IndexOutOfBoundsException` fırlatır (küçük bir Java denemesiyle doğrulandı). Doğru denetim `dugmeNo < 0 || dugmeNo >= acKapaDugmeleri.size()` biçiminde olmalıdır.

- Iterator kalıbı: amaç ve iki çözüm (ekrandan, 00:30:00 ve 00:32:10)
  - Güncel programlama dillerinde hazır bulunan bir kalıptır; çoğu dilde kendiniz kurmak zorunda değilsiniz, ama nasıl çalıştığını bilmek farklı bir bakış açısı kazandırır.
  - Amaç: parça-bütün yapısındaki ya da farklı parçalardan oluşan bir bileşenin parçalarını, bileşenin iç yapısını açığa çıkarmadan istenen biçimde dolaşmak. Dolaşırken yalnız izin verilen işlerin yapılması da sağlanır.
  - Motivasyon: generic yapı yokken (eski C++ standartlarında şablon sınıf yoktu; tüm sınıfların ortak üst sınıfı da yok) tam sayı listesi, öğrenci listesi gibi her tür için ayrı dolaşma kodu yazmak gerekir; yalnız tür adı değişir.
  - Java'nın iterator'ı kalıptaki yapıdan daha basittir. Basitlik kötü değildir; KISS ilkesi (Keep It Simple, Stupid): bir işi olduğundan karmaşık yapmak yanlıştır.
  - 1. çözüm: `List` (`Count()`, `Append(Element)`, `Remove(Element)`) ve onu dolaşan `ListIterator` (`First()`, `Next()`, `IsDone()`, `CurrentItem()`, `index`). Kullanım: `while (!IsDone()) Next()`.
    * Güçlü yönü: veri yapısı ile dolaşım biçimi birbirinden soyutlanmıştır.
    * Zayıf yönü: belli bir veri yapısı ile belli bir dolaşım biçimi arasında bağlılık oluşmuştur. Çift bağlı liste geri gidebilir, dairesel listede son elemandan başa dönülür, ikili ağaçta LDR, DLR, LRD gibi dolaşma varyasyonları vardır, tabloda (anahtar-değer) sıra önemli olmayabilir.
    * Bağlılığı azaltmak için genel bir liste yapısı arayüzü ve genel bir dolaşım arayüzü oluşturulabilir.
  - 2. çözüm (ekrandan, 00:32:10): `Client`, soyut `AbstractList` (`CreateIterator()`, `Count()`, `Append(Item)`, `Remove(Item)`) ve soyut `Iterator` (`First()`, `Next()`, `IsDone()`, `CurrentItem()`) üzerinden çalışır. `List` ve `SkipList`, `AbstractList`'ten; `ListIterator` ve `SkipListIterator`, `Iterator`'dan türer. Her somut liste kendi somut iterator'ını oluşturur, somut iterator da listeyi bilir.
    * SkipList: her elemanı dolaşmak yerine belli kriterlere uyan elemanları dolaşan bir yapı olarak örneklendi. Somut iterator'ın ek özelliklerine gerçekten ihtiyaç varsa tür denetimi ve tür dönüşümü (type casting) yapılır; bu, iç mantıkta değil kullanımda olduğu için büyük sorun değildir.

- Iterator kalıbının yapısı ve gerçekleme ayrıntıları (ekrandan, 00:40:00)

```
Client ⟶ Aggregate {CreateIterator()}           (soyut)
Client ⟶ Iterator {First(), Next(), IsDone(), CurrentItem()}   (soyut)
ConcreteAggregate {CreateIterator()} ⟶▷ Aggregate
   CreateIterator():  return new ConcreteIterator(this)
ConcreteIterator ⟶▷ Iterator
```

```
ConcreteAggregate ⇢ ConcreteIterator          (oluşturur)
ConcreteIterator ⟶ ConcreteAggregate          (dolaştığı bileşiği bilir)
```

  - Aggregate/ConcreteAggregate: parçaları arasında dolaşılacak nesnenin arayüzü/gerçeklemesi. Iterator (gezgin)/ConcreteIterator: dolaşma işleminin arayüzü/gerçeklemesi. `java.util` paketindeki veri yapıları bu kalıbı (daha basit biçimiyle) kullanır.
  - Dolaşılan parçanın durumunun değiştirilmesine izin verilecek mi? İzin verilmeyecekse iterator parçanın kendisini değil bir kopyasını döndürebilir; yapılan değişiklik bileşik varlığın durumunu değiştirmez.
  - Dolaşma algoritmasını kim gerçekleyecek ve dolaşılan nesneler istekçiye nasıl verilecek (teker teker mi, hepsi birden, ör. dolaşılmış bir dizi olarak mı)? Dolaşma sırasını istekçi (client) de, gezgin de belirleyebilir. Farklı dolaşma biçimleri varsa client bunu iterator'ın bir enum üyesini ayarlayarak ya da isteği aggregate'e vererek (o da uygun iterator'ı döndürür) belirtebilir.

- Iterator örneği: iki yemek firmasının menüleri (ekrandan, 00:50:00)
  - Kaynak: Head First Design Patterns. Online yemek siparişi işine girmek için kahvaltılıkta uzman (Pancake House) ve yemekte uzman (Diner) iki firma alınıyor; menüler ortak kod üzerinden müşteriye sunulmak isteniyor. İleride yeni firmalar alınacak; vejetaryen menü de istenecek.
  - İki firma ortak `MenuItem` (ad, açıklama, vejetaryen mi, fiyat) kullanıyor (farklı olsaydı araya adapter konurdu). `PancakeHouseMenu` menüyü `ArrayList` ile tutup `getMenuItems()` ile `ArrayList` döndürüyor; `DinerMenu` dizi ile tutup dizi döndürüyor.
  - Hızlı ve kirli çözüm (`WaitressQuickAndDirty`): her firma için ayrı döngü (biri `size()`/`get(i)`, diğeri `length`/`[i]`). Firma ve seçenek sayısı arttıkça kod tekrarlanır, bakım kabusuna döner; kendini `if-else-else` zincirinde bulmak kötü tasarımın en önemli göstergelerindendir.
  - Iterator ile çözüm: «`interface`» `Menu` (`+ createIterator(): Iterator<MenuItem>`) ve «`interface`» `Iterator<MenuItem>` (`+ hasNext(): boolean`, `+ next(): MenuItem`). `DinerMenu` ve `PancakeHouseMenu`, `Menu`'yü; `DinerMenuIterator` ve `PancakeHouseMenuIterator`, Java'nın `Iterator` arayüzünü gerçekler. `Waitress` yalnız arayüzleri kullanır; menünün dizi mi liste mi olduğunu bilmez.
  - `DinerMenuIterator` dizideki mevcut konumu tutar; `hasNext` dizinin sonuna gelinmişse `false` döndürür, `next` o konumdaki elemanı döndürüp konumu bir artırır. Listeden eleman silmek (`remove`) kolay, dizinin ortasından silmek ise sonraki elemanları kaydırmayı gerektirir.
  - Karşılaştırma (ekrandan, 01:00:00): kalıplı `printVegetarianMenu`, her menünün iterator'ını ortak bir `private void printVegetarianMenu(Iterator<MenuItem> iterator)` metoduna verir; metot `while (iterator.hasNext())` döngüsünde `isVegetarian()` olanları yazdırır. Kalıpsız sürümde aynı döngü her firma için ayrı yazılmıştır.
  - Müşterinin gördüğü sonuç ve performans aynıdır; kazanç yeni seçenek ve firma eklemenin kolaylaşmasıdır. Bu örnekte kalıbı kurma çabası, kodu çekidüzen vermek için zaten harcanacak çabaya denk düşüyor; her zaman böyle olmayabilir.

- Memento kalıbı: amaç (ekrandan, 01:20:00)
  - Adı, kısa süreli hafızasını yitiren bir adamın kendine küçük notlar yazdığı "Memento" filmini hatırlatır: anı, o anın bütün resmini gösteren bir fotoğraf karesi gibi.
  - Amaç: bir nesnenin durum bilgisini sarmalama (encapsulation) ilkesini bozmadan elde etmek. Böylece nesneler ikincil saklama ortamında saklanabilir, geri yüklenebilir, ağ üzerinden aktarılabilir (Java'daki serialization/deserialization gibi); son yapılan işlemi geri alma (undo) düzeneği kurulabilir. Durum bilgisinin bir kısmını saklamak da yeterli olabilir.
  - Sorun: private üyelere açık erişim vermeden bu nasıl yapılır? Saklayan nesnenin durumu değiştirip saklamadığından (sahtecilikten) nasıl emin olunur? Örnek: banka hesabında para çekilip bakiye düşülmemiş bir durum saklanıp geri yüklenirse "havadan para" kazanılır.
  - Her sınıfa undo/redo kodlamak ya da bu iş için bir üst sınıftan kalıtım almak istenmez; Java ve C# gibi dillerde tek kalıtım hakkı vardır ve bu değerli hak iş mantığı için saklanmalıdır.

- Memento kalıbının yapısı (ekrandan, 01:30:00)

```
Originator {SetMemento(Memento m), CreateMemento(), state} ┄┄► Memento {GetState(), SetSta
   CreateMemento():  return new Memento(state)
   SetMemento(m):    state = m->GetState()
```

  - Originator: durum bilgisi saklanacak nesne. Memento: durum bilgisinin gerekli kısımlarını saklar. Caretaker: gerekli anlarda durum bilgisinin bir kopyasını alır ve elindeki kopyalardan birini geçerli durum hâline getirir (ör. undo/redo için birden fazla memento tutabilir).
  - Kritik gerçekleme ayrıntısı: Memento'nun metotlarına (özellikle `SetState`) yalnız Originator erişebilmeli; Caretaker yalnız saklamalı, sıfırdan memento da üretememeli.
  - C++'ta: Memento metotları private yapılır, Originator Memento'nun `friend`'i yapılır.
  - Java'da paket çözümü: Memento ve Originator aynı pakete, Caretaker başka pakete konur; metotlar paket erişimli olur. Pakete sonradan sınıf eklenmemesi için paket (jar) `sealed` olarak işaretlenir. Hoca bu çözümü pek sevmediğini söyledi.
  - Hocanın önerisi: Memento'yu Originator'ın iç sınıfı yapmak.

- Memento: iç sınıf ile çözüm (ekrandan, 01:38:40 ve 01:50:00)

```java
package dp.mementoV2.innerClass;
public class Originator {
   private State state;
   public Originator( ) { state = new State( ); }
   public Memento createMemento( ) {
      Memento m = new Memento( );
      m.setState(state.clone());
      return m;
   }
   public void setMemento( Memento m ) { state = m.getState(); }
   public void modify( String str ) { state.setDurum(str); }
   public String toString() { return state.getDurum(); }

   public class Memento {
      private State state;
      private State getState() { return state; }
      private void setState(State state) { this.state = state; }
   }
}
```

```java
package dp.mementoV2Outer;
import dp.mementoV2.innerClass.*;
import dp.mementoV2.innerClass.Originator.Memento;

public class Caretaker {
   public void test( ) {
      Originator subject = new Originator( );
      subject.modify("ilk durum");
      Memento mem = subject.createMemento( );
      subject.modify("yeni durum");
      //bir süre sonra...
      subject.setMemento( mem );
      System.out.println(subject);
   }
   public static void main( String[] args ) {
      Caretaker ct = new Caretaker();
      ct.test();
   }
}
```

  - `State` sınıfı saklanacak durumu (burada bir metin) sarmalar; `Cloneable`'ı gerçekler ve kendi `clone` metodu vardır (deep/shallow copy kararı kendimizde kalsın diye). Klon kullanılmazsa izin verilen bir durum değişikliği mementoyu da etkiliyordu.

  - İç sınıfın metotları private olduğu için Caretaker çağıramaz; varsayılan yapıcı da yoktur ve iç sınıftan nesne oluşturmak için dış sınıftan nesne gerekir. Program çıktısında durum "ilk durum", "yeni durum", yeniden "ilk durum" olur.
  - Derste canlı denenen açık: Caretaker kendi `Originator fakeSubject`'ini oluşturup `modify("sahte durum")` der, `fakeSubject.createMemento()` ile memento alır ve `subject.setMemento(memFake)` çağırırsa subject sahte duruma geçer. Bu çalıştı (küçük bir Java denemesiyle de doğrulandı). Hoca bunu, Java'da private erişiminin nesne değil sınıf düzeyinde olmasıyla açıkladı.
  - Önerilen önlemler: Originator'a evrensel tekil kimlik (UUID) verip mementoya koymak ve `setMemento`'da kimlik eşleşmiyorsa reddetmek (ama kimliğin de ele geçirilip geçirilemeyeceği düşünülmeli); mementoların bakımını Originator'ın kendisine verip Caretaker'ın `setMemento` yerine `activateMemento(int)` gibi bir numarayla geri dönüş istemesi. Ayrı bir Caretaker'ın bakım yapması şartsa daha ayrıntılı düşünmek gerekir.

- Memento örneği: doküman undo/redo (ekrandan, 01:40:00)

```
Controller 1 ──► * Document {- currentState: DocumentMemento, - states: List<DocumentMemo
                              + undo(), + redo(), + anyOperation()}
                        Document 1 ──► * DocumentMemento {- documentState}
Not: Controller'ın DocumentMemento ile hiçbir şekilde ilişkisi yok.
```

```
public void undo() {
 position -= 1;
 currentState = states.get(position);
}
public void redo() {
 position += 1;
 currentState = states.get(position);
}
public void anyOperation() {
 states.add(currentState);
}
```

  - Burada Document hem Originator hem Caretaker'dır; Controller (uygulama) yalnız undo/redo der. Caretaker ayrı olmadığı için hoca bunu "yarım memento" uygulaması olarak niteledi: bir ödünleşimdir, açıkça belirtilirse kabul edilebilir bir çözümdür.

- Observer kalıbı: amaç ve yapı (ekrandan, 02:10:00 ve 02:40:00)
  - Amaç: gözlenen (Subject) bir nesnenin durumu değiştiğinde onunla ilişkili bütün nesnelerin haberdar edilip güncellenmesi. Gözleyiciler gözleneni sürekli incelemesin; gözlenen değişince gözleyicileri haberdar etsin. Gözleme ilişkileri çalışma anında da kurulabilsin.
  - Örnek: hesap tablosundaki veri değişince ondan çizilen farklı grafiklerin (sütun, pasta) otomatik güncellenmesi; başka bir dosyadaki veriden grafik tanımlanabilmesi. (MVC örneğinde de geçmişti.)

```
Subject {Attach(Observer), Detach(Observer), Notify()} ──observers──►* Observer {Update()
   Notify():  for all o in observers { o->Update() }
      △                                                    △
ConcreteSubject {GetState(), SetState(), subjectState} ◄──subject── ConcreteObserver {Upda
   GetState():  return subjectState                        Update():  observerState = subj
```

  - Subject: gözlenme ilişkisini yöneten metotları (attach, detach, notify) içerdiği için arayüz değil soyut sınıf olması en mantıklısıdır. Observer bir arayüz olabilir; ama gözleyici de gözlenenleri tutacaksa ona da attach/detach eklenir ve soyut sınıfa döner.
  - Somut gözleyiciden somut gözlenene olan `subject` ilişkisi, belli bir somut gözleyicinin belli bir somut gözleneni izlemesini sağlar. Her somut gözleyici her somut gözleneni izleyebilsin isteniyorsa bu ilişki soyut Observer'dan soyut Subject'e taşınır. Hangisi işe yarıyorsa o seçilir.
  - ConcreteSubject: durumu değişince gözleyicilerini Observer arayüzüyle bilgilendirir. ConcreteObserver: gözlediği nesneyi gösteren bir işaretçi tutabilir.

- Observer gerçekleme ayrıntıları (ekrandan, 02:13:00 ve 02:20:00)
  - Pull model: gözlenen yalnız "değiştim" der, gözleyiciler ilgili durum bilgisini gözlenenden alır. Artıları: gözlenenin gözleyicilere bağlantısı azalır (düşük bağlaşım); gözleyici durum bilgisinin yalnız istediği kısmını çekebilir; haberleşme masrafı azalır (megabaytlarca durum taşımak yerine basit bir ileti ve gerekli getter çağrıları). Eksisi: gözleyici nelerin değiştiğini kendisi çıkarmak zorundadır, kodu karmaşıklaşır.
  - Push model: gözlenen, gözleyicilere değişiklik hakkında ayrıntılı bilgi (genelde tüm durumu) gönderir. Gözleyici kodu basitleşir; farklı değişiklikler için farklı `update` metotları olabilir. Gözlenen karmaşıklaşır; durumu değiştiren her metoda haber verme kodu eklenmelidir.
  - Aynı türden gözlenenlerin hepsinin gözleyicileri ortak olacaksa gözleyiciler statik bir veri yapısında tutulabilir.
  - Bir gözleyici birden fazla nesneyi gözleyebilir; gözlenendeki farklı tür değişiklikleri farklı tür gözleyiciler izleyebilir (ör. Java'daki listener mekanizması: klavye dinleyicisi yalnız klavye olaylarını alır).
  - Haber verme mesajını kim tetikleyecek? Hocanın anlatımı: gözleyici tetiklerse gözleneni belli aralıklarla sorgular; gözleneni kodlayan rahat eder ama gözlenen seyrek değişiyorsa sık sorgu verimsizdir ve bant genişliği harcar. Gözlenen tetiklerse peş peşe değişikliklerde gereksiz bildirimler gidebilir; kısa aralıklı değişikliklerde yalnız sonuncusunu göndermek gibi iyileştirmeler gözlenende kodlanabilir. Slaytta: gözleyiciler tetiklerse istekçileri kodlayan kişi tetikleme komutunu vermeyi unutabilir; gözlenen tetiklerse peş peşe değişiklikler tek mesaj yerine birçok mesaja yol açabilir.
  - Kullanım anları: bir soyutlamanın birbirine bağımlı birden fazla parçası olduğunda; bir nesnedeki değişikliklerin derleme anında bilinmeyen sayıda nesneyi etkilemesi gerektiğinde; bağlaşımı artırmadan bir nesnenin diğerlerini haberdar etmesi istendiğinde. Kalıp bağlaşımı çok az artırır: tek bir çoklu sahiplik ilişkisi ekler. Bir varlık hem gözleyici hem gözlenen olabilir.
- Observer örneği: müşteri ilişkileri programı (ekrandan, 02:26:30 ve 02:30:00)
  - Problem: yeni müşteri eklendiğinde "hoş geldiniz" e-postası gönderilecek ve ikamet adresinin doğruluğu denetlenecek.
  - Çözüm 1: davranışlar sabit kodlanır. `Customer.addCustomer()` hem `WelcomeLetter`'ı hem `AddrVerification`'ı çağırır. Peki ya gereksinimler değişirse? Yeni gereksinim: müşteri bir şubeye 10 km yakınsa mektuba yalnız o mağazada geçerli indirim kuponu eklenecek.
  - Çözüm 2: Observer kalıbı. `Customer`: `- myObs: Vector<ICObserver> {static}`, `+ attach(ICObserver) {static}`, `+ detach(ICObserver) {static}`, `+ getState()`, `+ notifyObs()` (gözleyicilerin `update`'ini çağırarak haberdar eder). «`interface`» `ICObserver`: `+ update(myCust: Customer)`. Gerçekleyenler: `AddrVerification`, `BrickAndMortar` (fiziksel mağaza; indirim kuponu), `WelcomeLetter`.
  - Yalnız müşteri nesneleri izlenecekse soyut bir Subject üst sınıfı hazırlamaya gerek kalmadan kalıp kullanılabilir. Gözleyiciler bütün müşteriler için ortak olduğundan statik tutulmuştur. Yeni bir kampanya ya da gereksinim yeni bir somut gözleyici yazılarak eklenir.

Hocanın Özellikle Vurguladığı Kısımlar

- Command'da kullanılmayan parametre
  - Receiver'ı `Execute`'a parametre olarak eklemek, parametrenin çoğu durumda kullanılmayacağı yerlerde tasarım zayıflığıdır; `setReceiver` ya da farklı yapıcılar tercih edilir.
- Parça-bütün ilişkisinde parçanın bütünü bilmesi
  - Kalıp kurarken parçaların bütünü de bilmesi gerekebilir (açma komutu açtığı dokümanı uygulamaya ekler).
- `if-else` zinciri kötü tasarım göstergesidir
  - Firmaya göre farklı kod çalıştıran zincirler yeni firma ve seçenek eklendikçe bakım kabusuna döner.

- KISS ilkesi
  - Java'nın basit iterator'ı işinizi görüyorsa onu kullanın; ihtiyaç daha karmaşıksa kalıbı kurun.
- Iterator ile Adapter farkı
  - Adapter yalnız bir metot çağrısına köprüdür; Iterator bileşik bir varlığın parçaları arasında eş biçimli dolaşmayı da sağlar. Farklı kompozit varlıkların eş biçimde dolaşılması gereken bir soruda Iterator kalıbını anmak gerekir.
- Memento'da gizlilik
  - Memento'nun durumunu yalnız Originator ayarlayabilmeli; Caretaker sahte memento üretememeli. Bu, dile göre farklı mekanizmalarla (C++ `friend`, Java paket + `sealed`, iç sınıf) sağlanır ve tam güvence kolay değildir.
- Observer'da gerçekleme kararları
  - Az genel öğüt, çok gerçekleme ayrıntısı veren bir kalıptır: önce pull/push modeline, sonra bildirimi kimin tetikleyeceğine, kısmi ya da tüm durum aktarımına karar verilmelidir.
- Tasarım sorusunda beklenenler
  - Doğru kalıbı seçmek ve nedenini mantıklı açıklamak; UML sınıf şemasında kalıbın gereksinimlerinin (ör. Observer'da kalıtım ilişkisi, Subject'ten Observer'a ok) görünmesi; genel tasarım kurallarına uyulması (üye alanlar private, üye metotlar public, kritik metotların uygun görünürlükte olması). Hoca, bir problemde birden fazla kalıbın birlikte gerekebileceğini ve gerekçesi ikna edici olan farklı bir kalıbın da kabul edilebileceğini söyledi.

Kısa Tekrar Notları

- Command: metot çağrısını nesneye çevirir; kuyruklama, günlük tutma, menü/düğmeye komut bağlama için kullanılır. Bileşenler: Client, Invoker, Command (`Execute`), ConcreteCommand, Receiver (`Action`).
- Akıllı ev örneği: `AcKapa` arayüzü (`ac`, `kapa`), üç somut düğme sınıfı, `Kumanda` (düğme listesi), `AnaProgram`.
- Iterator: bileşiğin parçalarını iç yapıyı açmadan dolaşır. Bileşenler: Aggregate (`CreateIterator`), ConcreteAggregate, Iterator (`First`, `Next`, `IsDone`, `CurrentItem`), ConcreteIterator.
- Iterator kararları: dolaşılan parçanın değiştirilmesine izin, dolaşma algoritmasını kimin gerçekleyeceği, elemanların teker teker mi toplu mu verileceği.
- Menü örneği: dizi ve `ArrayList` kullanan iki menü, ortak `Menu` ve `Iterator<MenuItem>` arayüzleriyle `Waitress`'a eş biçimli sunulur.
- Memento: sarmalamayı bozmadan durum saklama ve geri yükleme. Bileşenler: Originator (`CreateMemento`, `SetMemento`), Memento (`GetState`, `SetState`), Caretaker.
- Memento gizliliği: C++'ta `friend`, Java'da aynı paket + `sealed` jar ya da iç sınıf. İç sınıf çözümünde başka bir Originator'dan alınan memento ile sahte durum yüklenebildiği görüldü.
- Doküman undo/redo örneği: Document hem Originator hem Caretaker ("yarım memento").
- Observer: gözlenen değişince gözleyiciler haberdar edilir. Bileşenler: Subject (`Attach`, `Detach`, `Notify`), ConcreteSubject, Observer (`Update`), ConcreteObserver.
- Pull model: düşük bağlaşım, seçici veri çekme, karmaşık gözleyici. Push model: basit gözleyici, karmaşık gözlenen.
- Müşteri örneği: `Customer` statik `ICObserver` listesi tutar; `AddrVerification`, `BrickAndMortar`, `WelcomeLetter` gözleyicidir.

Detaylı Açıklamalar

- Command kalıbının özü, "ne yapılacağı" bilgisini bir nesneye koymaktır. Menü seçeneği ya da düğme yalnız bir `Command` nesnesi tutar ve tıklanınca onun `Execute` metodunu çağırır; hangi işin yapılacağını bilmez. Bu sayede aynı komut hem menüye hem düğmeye bağlanabilir, komutlar kuyruğa konabilir ya da günlüğe yazılabilir. Komutun gerçek işi yaptıracağı nesne (Receiver) bazen komutu tetikleyenle aynıdır (dosya açma örneğinde uygulama), bazen farklıdır; farklı olabilecekse Receiver'ın komuta verilme yolu tasarımda

düşünülmelidir. Hoca, bunu `Execute`'a parametre ekleyerek yapmanın, parametre çoğu zaman kullanılmayacaksa zayıf bir tasarım olduğunu; bunun yerine `setReceiver` metodu ya da somut komutlarda farklı yapıcılar kullanılabileceğini anlattı.

- Akıllı ev örneğinde her düğme bir komut nesnesidir. `Kumanda` (Invoker) düğmelerin ne olduğunu bilmez, yalnız `AcKapa` arayüzünü çağırır. Aydınlatma, elektronik alet ve iklimlendirme cihazının açılıp kapanması gerçek sistemde farklı kodlarla yapılacağı için ayrı somut sınıflar vardır. Örnek bilerek basit tutulmuştur (`ac` ve `kapa` her zaman `true` döndürür); hoca örneğin, bir termometre ölçümünü geri döndüren bir komutla, yani Receiver eyleminin sonuç döndürmesiyle geliştirilebileceğini not etti.
- Iterator kalıbı, veri yapısı ile onu dolaşma biçimini ayırır. İlk çözümde `List` ile `ListIterator` ayrılmış ama birbirine bağlı kalmıştır: liste dolaşılacaksa mutlaka `ListIterator`, ağaç dolaşılacaksa `TreeIterator` gerekir. Yapılacak iş yalnız "sıradaki eleman var mı, varsa getir" ise bu ayrım istemciye gereksiz yük getirir. İkinci çözümde istemci yalnız soyut `AbstractList` ve soyut `Iterator`'ı bilir; her somut liste kendi somut iterator'ını `CreateIterator` ile üretir. Kalıp bazı kararları açık bırakır: dolaşırken parçanın değiştirilmesine izin verilip verilmeyeceği (verilmeyecekse kopya döndürülür), dolaşma algoritmasının aggregate'te mi iterator'da mı olacağı ve elemanların teker teker mi yoksa toplu mu verileceği.
- Yemek menüsü örneğinde iki firmanın menüsü farklı veri yapılarıyla (dizi ve `ArrayList`) gerçeklenmiştir. Kalıpsız `Waitress` her firma için ayrı döngü yazar; vejetaryen menü, yalnız kahvaltı, yalnız öğle yemeği gibi seçenekler ve yeni firmalar eklendikçe bu döngüler çoğalır. Kalıplı sürümde her menü `createIterator()` ile bir `Iterator<MenuItem>` döndürür; `Waitress` tek bir `printMenu(Iterator)` ya da `printVegetarianMenu(Iterator)` metoduyla her menüyü aynı biçimde dolaşır. Java'nın kendi `Iterator` arayüzü kullanılmış, `hasNext` ve `next` her menü için özelleştirilmiştir. Hoca, bu örnekte kalıbı kurmanın ek bir çaba gerektirmediğini, kodu toparlamak için zaten harcanacak çabaya denk düştüğünü vurguladı.
- Memento kalıbı, bir nesnenin durumunu dışarıda saklamanın sarmalamayı bozma riskine karşı geliştirilmiştir. Bütün üyeleri public yapmak ya da getter/setter açmak saklayıcıya durumu değiştirme fırsatı verir (banka hesabı örneği). Kalıpta durumu Originator kendisi bir Memento'ya yazar ve yine kendisi geri okur; Caretaker yalnız mementoları saklar ve gerektiğinde geri verir. Bunun güvencesi dile özgü mekanizmalara bağlıdır: C++'ta `friend`, Java'da paket erişimi ve `sealed` jar. Hoca daha temiz bir yol olarak Memento'yu Originator'ın iç sınıfı yaptı; iç sınıfın private metotlarına dış sınıf erişebilir, Caretaker erişemez. Derste canlı denemede, Caretaker'ın ikinci bir Originator oluşturup ondan aldığı mementoyu asıl nesneye yükleyebildiği görüldü. Hoca bunun için Originator kimliğiyle (UUID) doğrulama ya da mementoların bakımını Originator'a verip Caretaker'ın yalnız numarayla geri dönüş istemesi gibi önlemler önerdi; ayrı bir Caretaker şartsa sorunun daha ayrıntılı düşünülmesi gerektiğini söyledi.
- Observer kalıbında gözlenen (Subject), gözleyicilerinin (Observer) listesini tutar ve durumu değişince her birinin `Update` metodunu çağırır. En önemli karar, durum bilgisinin nasıl aktarılacağıdır. Pull modelde yalnız "değiştim" bilgisi gider, her gözleyici ilgilendiği kısmı kendisi çeker; bağlaşım ve haberleşme masrafı azalır ama gözleyici neyin değiştiğini bulmak zorundadır. Push modelde gözlenen ayrıntılı bilgi gönderir; gözleyici basitleşir, gözlenen karmaşıklaşır. Bildirimin kim tarafından tetikleneceği de ayrıca düşünülmelidir. Müşteri örneğinde her yeni müşteri eklenmesi, statik bir gözleyici listesindeki adres doğrulama, hoş geldin mektubu ve fiziksel mağaza kuponu gözleyicilerine bildirilir; yeni bir gereksinim yeni bir gözleyici sınıfıyla, `Customer`'a dokunmadan eklenir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.